COURS MAGISTRAL · GESTION COLLABORATIVE

Travailler à plusieurs sur du code avec Git et GitHub

Versionner, fusionner, relire, livrer : les bases d’un travail d’équipe traçable

---

M1 Data Science en santé · Université de Lille

Antoine Teston · antoine.teston@univ-lille.fr · CM de 2 h, suivi d’un TD de 3 h

OBJECTIFS

À la fin du CM et du TD, vous savez…

01 **Expliquer le contrôle de version**
et ce que Git apporte à une équipe

02 **Utiliser les commandes essentielles**
commit, branches, merge, push, pull

03 **Mener un projet sur GitHub**
pull requests, revue, résolution de conflits

04 **Mettre en place un workflow d’équipe**
règles, intégration continue (CI), versions

PLAN

Quatre temps, deux démonstrations

01	Pourquoi versionner son travail	15 min
02	Git : le modèle et les commandes · démo	45 min
03	Collaborer sur GitHub · démo	30 min
04	Un workflow d’équipe	20 min
→	Synthèse, questions, préparation du TD	10 min

01

Pourquoi versionner son travail

Du dossier « final_v3 » à l’historique partagé

Un dossier que tout le monde a connu

```
PROJET_COHORTE/
analyse_cohorte.ipynb
analyse_cohorte_v2.ipynb
analyse_cohorte_v2_corrige_Lea.ipynb
analyse_cohorte_final.ipynb
analyse_cohorte_final_v3.ipynb
analyse_cohorte_final_v3_VRAI.ipynb
```

Laquelle a produit la figure 2 du mémoire ?

Qu’a changé Léa, et pourquoi ?

Comment retrouver la version d’avant-hier ?

Et si deux personnes modifient le même fichier en même temps ?

Ce qu’apporte un gestionnaire de versions

Historique

Qui a changé quoi, quand et pourquoi : chaque version est datée, attribuée à son auteur, commentée.

Retour arrière

Revenir à n’importe quel état passé, ou annuler une seule modification.

Parallèle

Chacun avance sur sa branche ; l’outil combine les travaux.

Relecture

Toute modification peut être relue avant d’entrer dans la version commune.

EN DATA SCIENCE EN SANTÉ

Pouvoir dire quel code, dans quelle version, sur quelles données, a produit un résultat.

Reproduire
une analyse publiée, un mémoire, un indicateur

Justifier
un traitement de données de santé lors d'un audit

Transmettre
le projet d'un stagiaire parti, sans tout réécrire

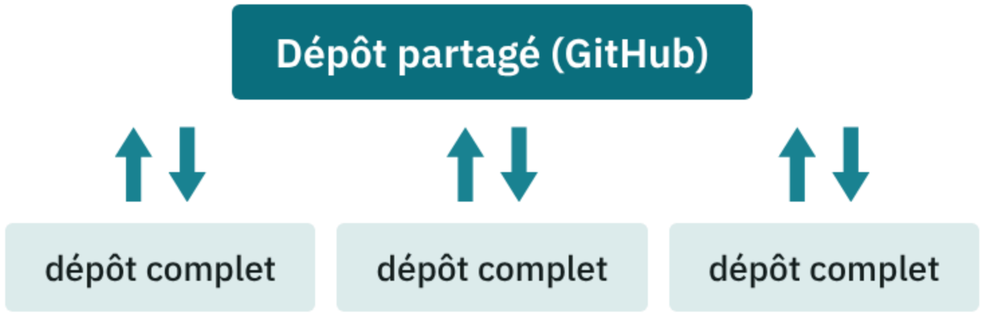
Centralisé ou distribué

CENTRALISÉ · SVN, CVS



Chaque commit part sur le serveur. Hors ligne ou serveur en panne : ni historique ni commit.

DISTRIBUÉ · GIT



Commit en local, partage explicite par push et pull. Chaque clone contient tout l'historique.

Git en bref

2005

créé par Linus Torvalds pour le développement du noyau Linux

Licence	Logiciel libre (GPLv2), multiplateforme
Version	2.56, publiée le 28 septembre 2026
À venir	Git 3.0, visé pour fin 2026 : branche initiale « main » et SHA-256 par défaut
Usage	Le standard de fait, du projet de stage au noyau Linux

Sources : git-scm.com (dernière version, consulté le 2 novembre 2026) ; notes de version de Git 2.52 et LWN sur la préparation de Git 3.0.

Git n’est pas GitHub

	Git	GitHub, GitLab, Bitbucket…
Nature	un logiciel	un service web
Où	sur votre machine	sur un serveur
Rôle	historique, branches, fusions	partage, pull requests, issues, intégration continue
Seul ?	fonctionne sans réseau	n’a de sens qu’avec Git

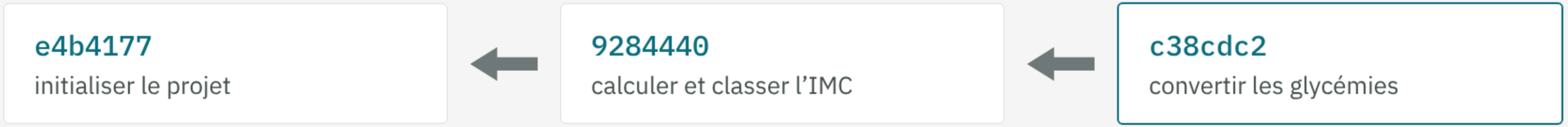
GitLab propose les mêmes fonctions, souvent installé sur les serveurs d’un hôpital ou d’une université. On y revient en partie 3.

02

Git : le modèle et les commandes

Commits, zones, branches, fusions, conflits, dépôt distant

Un commit est un instantané du projet



Chaque commit enregistre l'état complet des fichiers, pas seulement les différences.

Il pointe vers son **parent** : l'historique est une chaîne qu'on remonte.

Son identifiant est une empreinte de son contenu : on ne peut pas le modifier sans le remplacer.

```
$ git cat-file -p c38cdc2 (extrait)
tree dc862a7f6d1c…
parent 92844407bdd6…
author Antoine Teston <…> 1793010000 +0100
commiter Antoine Teston <…> 1793010000 +0100

feat: convertir les glycémies en mg/dL
```

Trois zones en local, une à distance

Répertoire de travail
vos fichiers, tels que vous les éditez

`git add` →

Index
ce qui partira dans le prochain commit

`git commit` →

Dépôt local
l'historique des commits, dans .git/

`git push` →

Dépôt distant
« origin », sur GitHub

Dans l'autre sens : git restore (répertoire), git restore --staged (index), git pull (distant vers local).

Pourquoi un index ? Pour choisir ce qui va ensemble : deux modifications sans rapport font deux commits.

Le cycle de tous les jours

```
git status                          # où en suis-je ?
git diff                            # qu'ai-je modifié ?
git add santestats/rapport.py       # je prépare le commit
git diff --staged                   # je relis ce qui partira
git commit -m "feat: afficher l'âge moyen"
git log --oneline --graph           # l'historique
```

Réflexe : git status avant et après chaque commande. Git dit presque toujours quoi faire ensuite.

Une fois par machine : git config --global user.name, user.email et pull.rebase false.

Un bon commit se lit dans six mois

Convention **Conventional Commits** : type: description, à l'infinitif.

type	usage
`feat`	nouvelle fonction
`fix`	correction de bug
`test`	tests
`docs`	documentation
`refactor`	réécriture à comportement égal
`chore`	configuration, CI

À ÉVITER

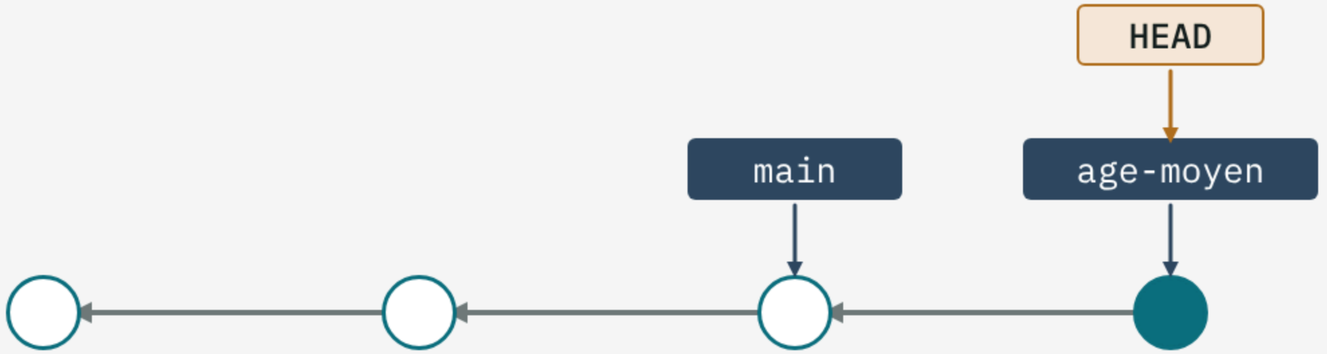
```
modifs · wip · correction · version finale ok
```

À PRÉFÉRER

```
fix: refuser une taille négative dans calculer_imc
feat: convertir les glycémies exprimées en g/L
```

Un commit = un changement cohérent. Le pourquoi va dans le corps du message, sous une ligne vide.

Une branche est un simple pointeur



git switch -c age-moyen, puis un commit : seule la branche courante avance.

Une branche désigne un commit. La créer est instantané, quelle que soit la taille du projet.

À chaque commit, la branche courante avance vers le nouveau commit ; les autres ne bougent pas.

HEAD indique où vous êtes : en général, la branche courante.

Créer, changer, fusionner, supprimer

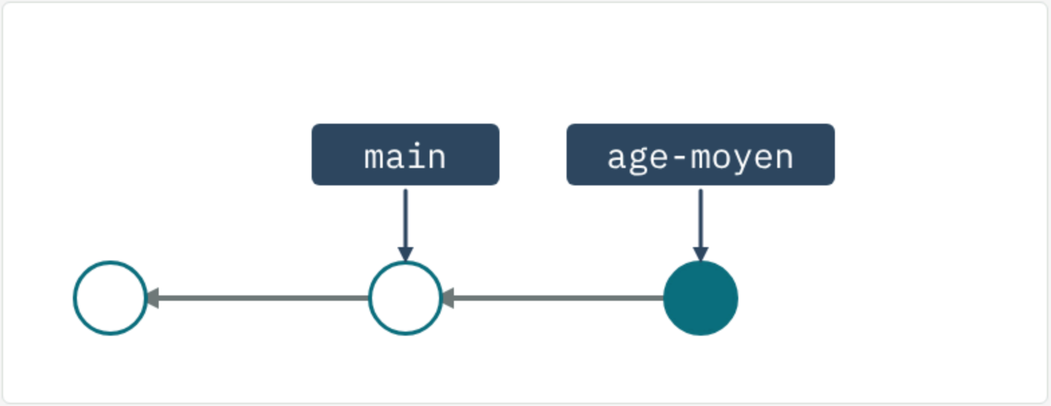
```
git branch                  # lister les branches locales
git switch -c age-moyen     # créer la branche et s'y placer
git switch main             # revenir sur main
git merge age-moyen         # fusionner age-moyen DANS main
git branch -d age-moyen     # supprimer une branche fusionnée
```

On fusionne toujours dans la branche courante. D'abord se placer sur la branche qui reçoit, puis git merge celle qui arrive.

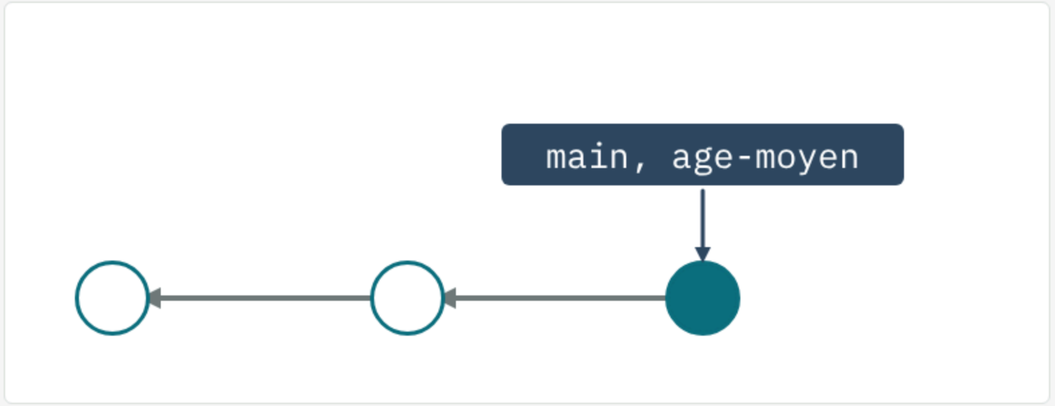
git switch et git restore (Git 2.23, 2019) remplacent les deux usages de git checkout, qu'on croise encore dans les tutoriels.

Fast-forward : Git avance le pointeur

AVANT

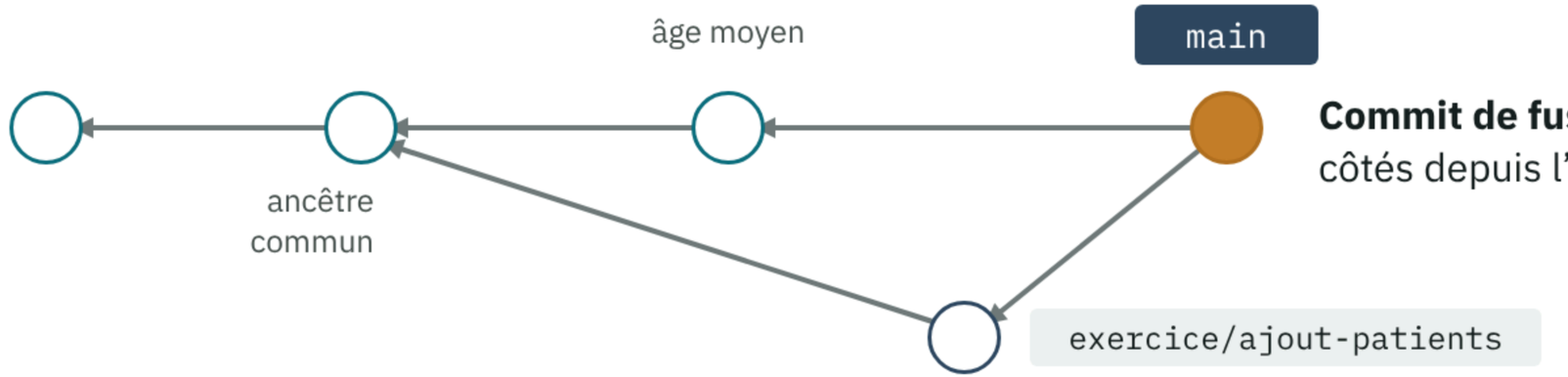


APRÈS git merge age-moyen



main n’a reçu aucun commit depuis la création de la branche : Git déplace simplement **main**, sans créer de commit. Message affiché : Fast-forward.

Fusion à trois sources : un commit à deux parents



Commit de fusion : il combine les deux côtés depuis l'ancêtre commun.

Les deux branches ont avancé chacune de leur côté : Git ne peut plus simplement déplacer un pointeur. Il compare chaque côté à l'ancêtre commun et crée un nouveau commit.

Le conflit : Git vous demande de trancher

Les deux branches ont modifié la même ligne de santestats/rapport.py :

```
<<<<<<< HEAD
TITRE = "Rapport de cohorte (santestats)"
=======
TITRE = "santestats cohort report"
>>>>>>> origin/exercice/titre-en
```

Au-dessus : votre branche (HEAD). En dessous : ce qui arrive.

1. git status : repérer « both modified »
2. éditer : garder la bonne version, supprimer les trois marqueurs
3. relancer les tests
4. git add santestats/rapport.py
5. git commit pour terminer la fusion

Paniquer est permis : **git merge --abort** remet tout comme avant.

DÉMONSTRATION · 10 MIN

Du clone au premier conflit

01 · git clone
récupérer le dépôt du TD

02 · git log --graph
lire l'historique et les branches

03 · commit
une branche, un commit, un fast-forward

04 · merge
fusion à trois sources

05 · conflit
titre-fr puis titre-en

06 · résolution
trancher, tester, git add, git commit

Mêmes commandes que la partie 1 du TD, sur le dépôt td-git-sante.

Synchroniser avec GitHub

Votre dépôt local

`main`

`origin/main`

origin/main : la dernière image connue de GitHub, mise à jour par fetch



git clone : une fois, copie tout

git fetch : récupère, sans toucher à main

git pull : fetch puis merge

git push : envoie vos commits

GitHub

`nom local : origin`

`main`

Première publication d'une branche : `git push -u origin age-moyen` ; ensuite, git push suffit.

Push refusé ? Récupérez d’abord

```
$ git push
 ! [rejected] main -> main (fetch first)
hint: Updates were rejected because the remote contains work that you do not have locally.
```

Par défaut

`git pull, puis git push`

fusionne le travail des autres avec le vôtre ; conflit possible

Variante

`git pull --rebase`

rejoue vos commits au-dessus des leurs : historique linéaire

Jamais

`git push --force`

sur une branche partagée : il efface le travail des autres

Ce qui n’entre jamais dans Git

```
.gitignore
# données et secrets
data/prive/
*.xlsx
.env
# fichiers générés
__pycache__/
.venv/
```

Données de patients réelles

Jamais, même dans un dépôt privé : données de santé, catégorie particulière du RGPD (article 9).

Secrets

Mots de passe, jetons, clés d’API : dans .env. Un secret poussé est un secret à révoquer.

Gros fichiers et résultats générés

On versionne le code qui les produit ; Git LFS ou DVC si un fichier lourd doit suivre.

L’historique n’oublie rien : supprimer le fichier dans un commit suivant ne l’efface pas des commits précédents.

Annuler sans rien casser

Je veux…	Commande	Réécrit ?
jeter une modification non indexée (définitif)	`git restore <fichier>`	non
retirer un fichier de l'index	`git restore --staged <fichier>`	non
corriger le dernier commit, pas encore poussé	`git commit --amend`	oui
annuler un commit déjà partagé	`git revert <id>`	non : nouveau commit
défaire des commits locaux	`git reset <id>`	oui
retrouver un commit « perdu »	`git reflog`	non

Sur une branche partagée, on ne réécrit pas l'historique : revert, pas reset ni amend.

03

Collaborer sur GitHub

Pull requests, revue de code, protection de main, intégration continue

Deux façons de contribuer

Collaborateurs

au TD

L'équipe est invitée sur un même dépôt. Chacun pousse ses branches ; main est protégée.

Pour : une équipe projet, un laboratoire, un service.

`Settings → Collaborators`

Fork

Une copie du dépôt sous votre compte. Vous y poussez, puis proposez une PR vers l'original.

Pour : contribuer à un projet open source sans droit d'écriture.

`bouton Fork`

La pull request : proposer, discuter, fusionner

Open **feat: convertir les glycémies exprimées en g/L #8**

`1-conversion-g-l` → `main` · 1 commit

Conversation Commits Checks Files changed

Ajoute g/L aux unités prises en charge. Closes #1

B a approuvé · tests : réussi

Merge pull request

1. une branche vers main
2. le pourquoi, et l'issue liée
3. le diff, commenté ligne par ligne
4. la revue et la CI
5. la fusion, par l'auteur

La revue de code

Ce qu’on vérifie

- le code fait ce que dit l’issue
- des tests couvrent le changement
- un collègue le comprend sans l’auteur
- aucune donnée réelle, aucun secret

Comment on commente

- sur la ligne concernée
- une question plutôt qu’un ordre
- bloquant ou simple suggestion : on le dit
- on conclut : Approve ou Request changes

`glycemie.py:8` Le test couvre 1,26 g/L ; que se passe-t-il pour « g/l » en minuscules ? (suggestion)

Ma pull request est en conflit

⚠ This branch has conflicts that must be resolved

```
git switch 2-seuil-hyperglycemie
git fetch origin
git merge origin/main      # fichiers en conflit
# trancher, retirer les marqueurs, pytest
git add CHANGELOG.md
git commit --no-edit
git push                   # la PR se met à jour
```

Pourquoi ? main a reçu une autre PR depuis la création de votre branche, sur les mêmes lignes.

On résout sur sa branche, jamais sur main : c'est main qu'on fait entrer dans la branche.

Le conflit touche le travail d'un collègue ? On en parle avant de trancher.

Protéger main par une règle

- ✓ pull request obligatoire avant toute fusion
- ✓ au moins une approbation d’un autre membre
- ✓ tests au vert (check « tests » requis)
- ✓ conversations résolues
- 🔒 suppression et push forcé bloqués

Source : documentation GitHub, « Available rules for rulesets » (docs.github.com).

`Settings → Rules → Rulesets → New branch ruleset`

Gratuit sur un dépôt public. Sur un dépôt privé : offres GitHub Pro, Team ou Enterprise (Pro inclus dans l’offre étudiante).

Les tests à chaque pull request

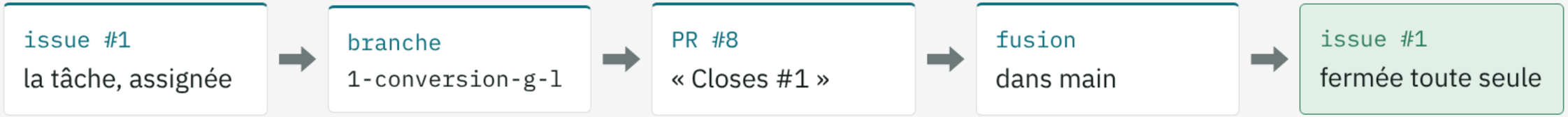
```
.github/workflows/tests.yml
name: tests
on:
  push: { branches: [main] }
  pull_request:
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
      - run: pip install -r requirements.txt
      - run: pytest -v
```

À chaque pull request, GitHub installe le projet sur une machine neuve et lance les tests.

Le résultat s'affiche dans l'onglet Checks de la PR.

Rendu obligatoire par la règle de main : une PR rouge ne se fusionne pas.

De l’issue à la fusion, un fil continu



Une issue décrit une tâche et son critère de fin. On l’assigne avant de commencer : chacun sait qui fait quoi.

Closes, Fixes ou Resolves suivi du numéro, dans la description de la PR : la fusion ferme l’issue.

DÉMONSTRATION · 10 MIN

Une pull request de bout en bout

`01 · issue`
la créer et se l'assigner

`02 · git push -u`
une branche, un commit, envoyés

`03 · pull request`
description et « Closes #n »

`04 · revue`
un commentaire sur une ligne, Approve

`05 · checks`
la CI lance les tests

`06 · fusion`
Merge, Delete branch, issue fermée

Le déroulé de la partie 2 du TD, sur un dépôt créé à partir du modèle td-git-sante.

Sur GitLab, même logique, autres mots

GitHub	GitLab
Pull request (PR)	**Merge request (MR)**
GitHub Actions · .github/workflows/	GitLab CI/CD · .gitlab-ci.yml
Rulesets, protection de branche	Protected branches, règles d’approbation
Issues, Projects	Issues, Boards
Fork	Fork
gh (ligne de commande)	glab (ligne de commande)

Les commandes Git ne changent pas : clone, commit, push, pull, merge sont identiques.

04

Un workflow d’équipe

Choisir une organisation, l’écrire, s’y tenir

Trois organisations courantes

GitHub flow

au TD

main toujours stable ; une branche courte par tâche ; fusion par PR relue.

Pour : petites équipes, projets d'analyse, livraison continue.

Git flow

En plus de main : develop, release et hotfix pour préparer et corriger les versions.

Pour : versions planifiées et validées, au prix d'un processus plus lourd.

Trunk-based

Tout le monde intègre dans main plusieurs fois par jour ; branches de quelques heures.

Pour : équipes rodées, avec une CI solide et les fonctions inachevées masquées (feature flags).

Le bon choix est celui que toute l'équipe applique : on l'écrit dans CONTRIBUTING.md.

Le GitHub flow en six étapes

01
Issue
une tâche, une personne

→

02
Branche
depuis un main à jour

→

03
Commits
petits, tests au vert

→

04
Pull request
tôt, avec Closes #n

→

05
Revue et CI
approbation, checks verts

→

06
Fusion
puis branche supprimée

```
git switch main && git pull && git switch -c 3-obesite-morbide
pytest && git commit -am "feat: distinguer l’obésité morbide"
git push -u origin 3-obesite-morbide # puis la PR sur GitHub
```

Fusionner une PR : trois boutons

Merge commit

```
*   Merge pull request #8
|\
| * test: cas g/L
| * feat: unité g/L
|/
```

Garde tout : chaque commit et la trace de la PR. Par défaut sur GitHub.

Squash and merge

```
* feat: unité g/L (#8)
* …
```

Un commit par PR : main se lit comme un journal ; le détail reste dans la PR.

Rebase and merge

```
* test: cas g/L
* feat: unité g/L
* …
```

Historique linéaire, commits conservés, mais réécrits : nouveaux identifiants.

Aucun n’est meilleur dans l’absolu : l’équipe en choisit un (Settings → General → Pull Requests).

Publier une version

MAJEUR.MINEUR.CORRECTIF

0.2.1	une correction
0.3.0	des fonctions nouvelles, compatibles
1.0.0	une rupture, ou la première version stable

```
git tag -a v0.2.0 -m "Version 0.2.0"
git push origin v0.2.0
```

Sur GitHub, une **Release** attachée à l'étiquette reprend les lignes du CHANGELOG.

En santé : on retrouve exactement le code d'un résultat : git switch --detach v0.2.0

Six règles d’équipe

- ✓ Une issue avant d’écrire du code
- ✓ Une branche courte par tâche, nommée d’après son issue
- ✓ Des PR petites, relues par quelqu’un d’autre
- ✓ main protégée : PR, approbation, CI verte
- ✓ Se mettre à jour souvent : les petits conflits se règlent vite
- ⚠ Jamais de push forcé sur une branche partagée, ni de données réelles

Ces règles s’écrivent dans CONTRIBUTING.md ; les plus importantes sont imposées par la configuration du dépôt.

SYNTHÈSE

À retenir

- Un commit est un instantané ; une branche, un pointeur.
- Le commit reste local ; le push partage.
- Un conflit est une décision humaine, pas une erreur.
- Tout entre dans main par une PR relue, CI au vert.
- Jamais de données réelles ni de secrets dans un dépôt.

POUR LA DISCUSSION

Pendant votre stage de M1, comment l'équipe partageait-elle son code ? Qu'auriez-vous changé ?

POUR LE TD · 3 H

Préparez votre poste maintenant

- ✓ un compte GitHub : à créer tout de suite sur github.com/signup
- ✓ rien d'installé ? GitHub Codespaces fournit Git, Python et gh dans le navigateur
- ✓ sinon : Git 2.40+, Python 3.10+, VS Code, gh connecté (gh auth login)
- ✓ git config --global : user.name, user.email, pull.rebase false

AU PROGRAMME

1. Git en local, seul, sur le dépôt td-git-sante
2. Par équipes de quatre : issues, pull requests, revues, vrais conflits
3. CI obligatoire et version 0.2.0

Noté individuellement : ce que GitHub enregistre à votre nom

Dépôt du TD : github.com/Lille-Health-Data-Science/td-git-sante

POUR ALLER PLUS LOIN

Ressources

Pro Git, en français	git-scm.com/book/fr/v2
Les branches, en jouant	learngitbranching.js.org
Documentation GitHub	docs.github.com/fr
Conventional Commits	conventionalcommits.org/fr
Versionnage sémantique	semver.org/lang/fr

Antoine Teston · antoine.teston@univ-lille.fr